

FITLIFE AI

# Documentacion de razonamiento y arquitectura

Análisis técnico del proyecto implementado: decisiones de diseño, flujos de información, seguridad, persistencia y límites del asistente.

|                                                                                                            |                                                                                                        |
|------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|
| <b>Alcance</b><br>Aplicación de fitness y nutrición personalizada con planes de 28 días y chat contextual. | <b>Versión observada</b><br>1.2.4 en FastAPI y frontend; análisis basado en el código del repositorio. |
| <b>Fecha</b><br>4 de octubre de 2026                                                                       | <b>Audiencia</b><br>Evaluación académica, desarrollo, operación y revisión de seguridad.               |

**Conclusión ejecutiva:** FitLife prioriza respuestas verificables desde datos persistidos. La generación de planes y las respuestas de rutina son deterministas; el modelo de lenguaje se usa sólo como apoyo contextual y queda detrás de guardrails.

Este documento no sustituye una auditoría de producción ni afirma que los servicios estén desplegados o activos. Describe el comportamiento que implementa el código revisado.

# 1. Resumen del sistema

FitLife AI es una aplicación web autónoma para crear y consultar planes de ejercicio y nutrición. El usuario crea una cuenta, captura un perfil, genera un calendario de 28 días, marca progreso y conversa con un asistente orientado estrictamente al dominio de bienestar.

## Capacidades implementadas

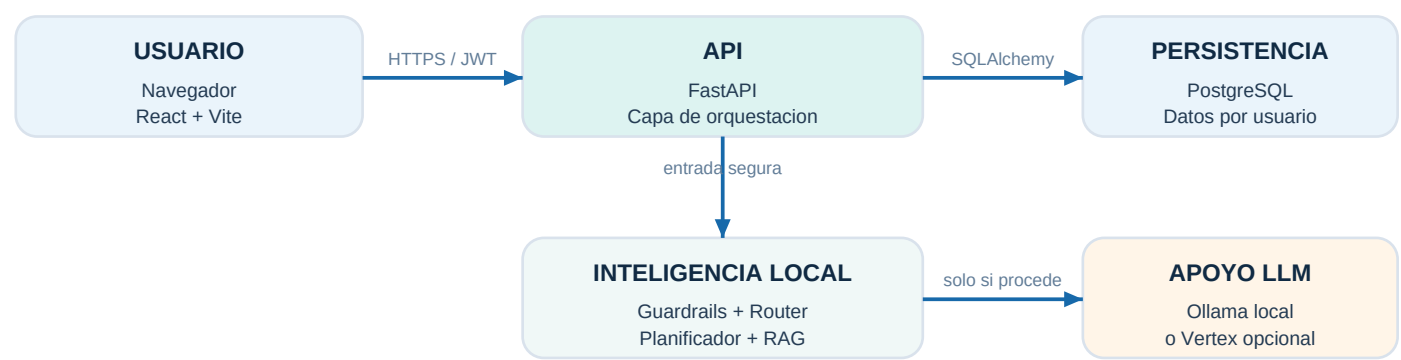
| Capacidad     | Cómo se resuelve                                                                             | Fuente de verdad              |
|---------------|----------------------------------------------------------------------------------------------|-------------------------------|
| Identidad     | Registro e inicio de sesión; contraseña derivada con scrypt y token firmado HS256.           | users y token vigente         |
| Perfil seguro | Validación de forma, rangos, reglas de compatibilidad y rechazo de campos extra.             | profiles validado en backend  |
| Plan          | Planificador local crea 28 jornadas desde objetivos, disponibilidad y restricciones.         | plans + plan_days             |
| Progreso      | Marca por fecha y plan; la API calcula el siguiente entrenamiento y el último completado.    | progress + completed_days     |
| Chat          | Guardrails, recuperación priorizada y respuesta contextual, con memoria compacta por cuenta. | plan, RAG local y user_memory |

## Objetivos de diseño

- Mantener aislados los datos de cada cuenta y no depender del historial visual del navegador para la memoria durable.
- Evitar que un modelo generativo invente qué ejercicios, comidas o progreso pertenecen al usuario.
- Ofrecer una experiencia local y desplegable con Docker: React/Vite, FastAPI, PostgreSQL, Ollama y un índice vectorial persistente.
- Bloquear en el borde solicitudes de inyección, jailbreak, riesgo y temas ajenos antes de alcanzar agentes, RAG o LLM.

## 2. Arquitectura de alto nivel

La arquitectura es una separación clara entre presentación, API, almacenamiento y capacidades de IA. El backend es el punto de control: aplica autenticación, reglas del perfil, guardrails, acceso por usuario y validación de salida.



Principio rector: el plan persistido determina rutina, comidas, progreso y pertenencia de ejercicios.

### Responsabilidades por capa

| Capa     | Responsabilidad                                                                                                      | Elementos observados                                      |
|----------|----------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Frontend | Recoge datos, muestra calendario y chat; conserva sólo sesión de UI y token durante la sesión del navegador.         | React, Vite, AuthContext, componentes Profile/Plan/Chat   |
| API      | Expone contratos REST, valida entradas, autoriza al usuario y orquesta consultas.                                    | FastAPI, Pydantic, dependencias current_user y db_session |
| Dominio  | Calcula planes, clasifica intención, busca ejercicios y resuelve preguntas de plan.                                  | agents/planner, fitness, nutrition, conversation y router |
| Datos    | Persiste entidades de negocio y memoria estructurada por usuario.                                                    | SQLAlchemy + PostgreSQL                                   |
| IA local | Recupera conocimiento con embeddings hash locales; consulta LLM sólo cuando la ruta no tiene respuesta determinista. | LocalVectorStore, LocalRetriever, LocalLLM                |

### Despliegue

docker-compose define PostgreSQL, Ollama, una inicialización que descarga el modelo, backend y frontend. Los volúmenes separan datos relacionales, modelo de Ollama e índice vectorial. Por defecto, la interfaz se publica en el puerto 5180 y la API en 8010.

### 3. Razonamiento detrás del diseño

El proyecto adopta una estrategia de **determinismo primero**. No trata al modelo de lenguaje como autoridad sobre los datos personales o el calendario. Esta decisión reduce errores de alucinación y hace que las respuestas críticas sean auditable.

#### Decisiones y justificación

| Decisión                      | Razonamiento                                                                                        | Consecuencia                                                                            |
|-------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| Planificador determinista     | Un plan de 28 días debe ser reproducible a partir del perfil y compatible con los días disponibles. | La lógica de ejercicios, comidas, calorías y descansos se materializa antes de chatear. |
| Plan persistido como verdad   | El chat no debe inferir pertenencia de un ejercicio ni progreso desde texto libre.                  | Las consultas de rutina consultan plan_days y progreso; el LLM no modifica esos hechos. |
| RAG local con hash embeddings | La base de conocimiento debe consultarse sin enviar documentos a un proveedor de embeddings.        | Índice simple, reproducible y persistente; apto para una base acotada.                  |
| Memoria compacta              | Retomar contexto no requiere conservar una conversación completa en la interfaz.                    | Se guardan último ejercicio, comida, workout, intención y tema por usuario.             |
| Adaptador de LLM              | La política de negocio no debe quedar acoplada a Ollama ni a un proveedor concreto.                 | LocalLLM y VertexLLM comparten interfaz answer(system, user).                           |

#### Orden de recuperación

La resolución favorece evidencia específica y de menor riesgo: (1) rutina persistida, (2) comidas persistidas, (3) base de conocimiento local/RAG y (4) LLM sólo si la pregunta lo requiere. Para una técnica de ejercicio, se combina la programación del usuario con una guía local cuando existe; para membresía o series, la rutina basta.

**Implicación: una respuesta puede ser útil aun cuando Ollama no esté disponible. Sólo las rutas que realmente requieren generación devuelven 503 con un identificador de error si el LLM falla.**

## 4. Flujo de razonamiento del chat

El endpoint POST /api/v1/chat contiene el flujo de decisión. El guardrail se evalúa antes de crear una respuesta de agentes, recuperar RAG o llamar al LLM. Después se aplica una validación de salida sobre la respuesta permitida.



El LLM no decide el contenido del plan ni recibe una consulta fuera de dominio.

### Casos principales

| Pregunta                                    | Ruta                                                                                    | Respuesta esperada                                                              |
|---------------------------------------------|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| ¿Qué me toca hoy?                           | Intención de plan -> estado calculado desde plan_days.                                  | Resumen de jornada, actividad y comidas del día.                                |
| ¿Cuántas series tiene la flexión inclinada? | Fitness -> búsqueda en todos los días del plan del usuario.                             | Programación exacta de la rutina; no depende del LLM.                           |
| ¿Cómo preparo esta comida?                  | Nutrition -> comida del plan o recuperación local; LLM sólo en un caso no determinista. | Contexto de alimentación sin inventar elementos del plan.                       |
| ¿En qué nos quedamos?                       | Seguimiento -> user_memory y contexto del plan.                                         | Referencia al último tema persistido, validando que el ejercicio siga presente. |
| Escríbeme código Python                     | Guardrail OFF_TOPIC.                                                                    | Bloqueo con explicación de alcance, antes de agentes/RAG/LLM.                   |

### Regla de excepción contextual

Una pregunta breve de seguimiento que no contiene palabras del dominio puede permitirse si el clasificador conversacional reconoce que se refiere al plan. Las señales de inyección, jailbreak o riesgo conservan precedencia y no se anulan con ese mecanismo.

## 5. Modelo de datos y contratos

SQLAlchemy modela las entidades. La relación lógica se centra en el usuario: perfil, plan, sesiones de chat, progreso y memoria se asocian a su identificador. Cada endpoint protegido resuelve `current_user` desde el bearer token antes de consultar datos.

### Entidades persistentes

| Entidad                       | Propósito                                            | Aislamiento / integridad                                                                     |
|-------------------------------|------------------------------------------------------|----------------------------------------------------------------------------------------------|
| users                         | Cuenta y hash de contraseña.                         | username único; el id es la clave de las demás entidades.                                    |
| profiles                      | Datos de perfil y versión.                           | Una fila por usuario; sólo el backend valida y persiste.                                     |
| plans / plan_days             | Plan activo y sus 28 jornadas JSON estructuradas.    | Un plan por usuario; sus días se consultan con <code>plan_id</code> .                        |
| progress                      | Ejecución por fecha, estado y sello temporal.        | Filtrado por <code>user_id</code> y <code>plan_id</code> ; no permite fechas ajenas al plan. |
| chat_sessions / chat_messages | Conversación persistida y trazabilidad de intención. | Una sesión debe pertenecer al usuario autenticado.                                           |
| user_memory                   | Contexto compacto de conversación.                   | Una memoria por usuario; vinculada opcionalmente al plan vigente.                            |

### Contratos REST relevantes

| Método     | Ruta                                                        | Propósito                                                          |
|------------|-------------------------------------------------------------|--------------------------------------------------------------------|
| POST       | <code>/api/v1/auth/signup</code> y <code>/login</code>      | Crea cuenta o inicia sesión y devuelve <code>access_token</code> . |
| GET / PUT  | <code>/api/v1/profile</code>                                | Lee o actualiza el perfil mediante un payload cerrado.             |
| POST / GET | <code>/api/v1/plan/generate</code> y <code>/plan</code>     | Regenera el plan activo o devuelve su calendario.                  |
| POST       | <code>/api/v1/progress</code>                               | Marca o desmarca una fecha perteneciente al plan.                  |
| POST / GET | <code>/api/v1/chat</code> y <code>/chat/{session_id}</code> | Envía una pregunta o recupera sólo el historial propio.            |
| GET        | <code>/api/v1/context</code>                                | Entrega memoria durable y estado resumido del plan.                |

## 6. Seguridad, seguridad de IA y privacidad

La seguridad se distribuye por capas. El frontend mejora la experiencia con validación inmediata, pero el backend repite toda validación relevante y no confía en que el cliente haya cumplido el contrato.

### Controles implementados

| Área          | Control                                                                      | Comportamiento                                                                   |
|---------------|------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Autenticación | script con salt aleatorio; JWT HS256 con expiración.                         | Contraseñas no se guardan en claro; una firma o expiración inválida produce 401. |
| Autorización  | current_user y verificaciones de propietario.                                | Un usuario no puede leer una sesión ni un día de plan de otra cuenta.            |
| Perfil        | Rangos, enums, restricciones compatibles y guardrail independiente de texto. | Los errores son 422 con field_errors; no se almacena un perfil inválido.         |
| Entrada IA    | Detección de inyección, jailbreak, solicitudes inseguras y fuera de dominio. | Se bloquea antes de agentes, RAG y LLM.                                          |
| Salida IA     | Filtro de información sensible e intentos OFF_TOPIC.                         | Sustituye una respuesta generada que viole la política.                          |
| Errores       | ID aleatorio para 500 y 503; registro estructurado de eventos.               | La interfaz recibe un mensaje controlado en lugar de la traza interna.           |

### Límites declarados

- FitLife ofrece orientación general de bienestar; no realiza diagnóstico médico ni atención de emergencia.
- La clasificación se basa en reglas y palabras clave: es deliberadamente explicable, pero puede requerir ampliación de vocabulario y pruebas de regresión al crecer el dominio.
- La implementación observada usa un secreto JWT de desarrollo como valor por defecto. Una puesta en producción debe obligar un secreto fuerte y una configuración gestionada.
- El chat persiste mensajes y memoria por usuario. Una política de retención, consentimiento y borrado de datos sería una extensión recomendable antes de una operación con usuarios reales.

## 7. Operación, pruebas y mantenibilidad

La aplicación puede levantarse de forma integrada con Docker Compose. El backend inicializa las tablas de forma idempotente y el recuperador crea su índice local al iniciar, utilizando la base de conocimiento incluida y, opcionalmente, una fuente pública normalizada.

### Evidencia de calidad en el repositorio

| Área              | Cobertura descrita                                                            | Ubicación                                 |
|-------------------|-------------------------------------------------------------------------------|-------------------------------------------|
| API y aislamiento | Contratos de endpoints, sesiones y separación por usuario.                    | backend/tests/test_api.py                 |
| Guardrails        | Bloqueo OFF_TOPIC antes del LLM y políticas de entrada/salida.                | backend/tests/test_guardrails.py          |
| Plan y fitness    | Plan de 28 días, disponibilidad, matching y regresiones de historial ruidoso. | backend/tests/test_planner_and_fitness.py |
| Perfil            | Matriz de validación y rechazo de payload no permitido.                       | backend/tests/test_profile_validation.py  |
| Frontend          | Reglas del perfil y contratos de perfil/chat.                                 | frontend/src/**/*.test.js                 |

### Observabilidad actual

El backend registra inicio, creación de cuenta, actualización de perfil, generación de plan, decisión de guardrail, resultado de chat y fallos inesperados. Los logs evitan incluir datos de salud de perfil en el mensaje de validación y usan user\_id, operación e identificadores de error para correlación.

### Mejoras priorizadas

| Prioridad | Mejora                                                                                          | Motivo                                                            |
|-----------|-------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| Alta      | Sacar secretos, URL de Vertex y configuración de proveedor a variables de entorno obligatorias. | Reduce exposición y permite despliegues por entorno.              |
| Alta      | Añadir migraciones versionadas y restricciones únicas para progreso por usuario/fecha/plan.     | Hace más robustas las actualizaciones y evita duplicados lógicos. |
| Media     | Medir latencia, tasa de bloqueos, errores LLM y calidad de recuperación.                        | Permite operar y mejorar el sistema basado en evidencia.          |
| Media     | Establecer retención, exportación y eliminación de chat/memoria.                                | Completa el manejo responsable de datos personales.               |

## 8. Trazabilidad del análisis

Las conclusiones de este documento proceden de la implementación revisada. Esta tabla facilita relacionar cada afirmación arquitectónica con el área de código correspondiente.

| Tema                      | Archivos revisados                                                  | Hallazgo                                                                        |
|---------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Entrada y orquestación    | backend/app/main.py                                                 | Endpoints, autenticación, orden de guardrails, contexto, plan, progreso y chat. |
| Planificador              | backend/app/agents/planner.py                                       | Construcción local de ejercicios, comidas, semanas y restricciones.             |
| Chat especializado        | backend/app/agents/{fitness,nutrition,conversation,router}.py       | Clasificación, búsqueda de ejercicio, nutrición y seguimiento.                  |
| Guardrails                | backend/app/guardrails/*.py                                         | Reglas de normalización, dominio, entrada, salida y perfil.                     |
| RAG                       | backend/app/rag/*.py y data/knowledge_base.json                     | Embeddings hash, índice JSON local y recuperación por similitud.                |
| Persistencia y seguridad  | backend/app/{db,security,config}.py                                 | Modelos SQLAlchemy, migración ligera, bcrypt, JWT y configuración.              |
| Interfaz                  | frontend/src/App.jsx, context, services y components                | Sesión, contratos API, formularios, calendario, progreso y widget de chat.      |
| Despliegue y verificación | docker-compose.yml, README.md, backend/tests, frontend package.json | Servicios, volúmenes, comandos y cobertura declarada.                           |

**Lectura final: la arquitectura está diseñada para que la personalización sea útil sin otorgar al LLM control sobre datos estructurados. El principal siguiente paso es reforzar configuración de producción y gobierno de datos, conservando el patrón actual de verdad persistida y validación por capas.**

Fin del documento.